Generative Artificial Intelligence

A Learner's Guide to GPT, DALL-E, Codex, Stable Diffusion, Transformers, GANs, Diffusion Models, and Enterprise Applications

Structured study notes

October 27, 2026

Contents

1 How to Use This Document **5**

2 The Big Picture: AI, ML, Deep Learning, and Generative AI **5**

2.1 Artificial Intelligence 5

2.2 Machine Learning 5

2.3 Deep Learning 6

2.4 Generative AI 6

2.5 Discriminative and Generative Models 6

3 Core Ideas Needed Before Studying Large Models **7**

3.1 Data, Parameters, and Training 7

3.2 Loss and Gradient Descent 7

3.3 Probability and the Softmax Function 8

3.4 Tokens and Tokenisation 8

3.5 Embeddings 8

4 Transformers: The Foundation of Modern Language Models **8**

4.1 What Is a Transformer? 8

4.2 The Intuition Behind Attention 9

4.3 Why Use Multiple Attention Heads? 9

4.4 Positional Information 9

4.5 A Simplified Transformer Block 10

4.6 Encoder, Decoder, and Encoder–Decoder Designs 10

4.7 How a GPT-Style Model Generates Text 10

4.8 Decoding: Greedy, Sampling, Temperature, and Top-p 11

4.9 What Transformers Do Not Automatically Guarantee 11

5 GPT: Generative Pre-trained Transformer **11**
5.1 Definition . . . 11
5.2 How GPT-Style Systems Are Developed . . . 12
5.3 What Happens When We Ask GPT a Question? . . . 12
5.4 Strengths and Limitations . . . 12
5.5 Prompting: Giving Clear Instructions . . . 13

6 DALL-E: Generating Images from Text **13**
6.1 Definition . . . 13
6.2 How Text-to-Image Generation Works at a High Level . . . 13
6.3 What Makes Text-to-Image Difficult? . . . 13

7 Codex: AI Assistance for Software Development **14**
7.1 Definition . . . 14
7.2 How a Code Model Works . . . 14
7.3 Example . . . 14
7.4 Risks of AI-Generated Code . . . 14

8 Stable Diffusion and Latent Diffusion Models **15**
8.1 Definition . . . 15
8.2 What Is a Diffusion Model? . . . 15
8.3 Forward Diffusion: Adding Noise . . . 15
8.4 Reverse Diffusion: Learning to Denoise . . . 15
8.5 Why Work in Latent Space? . . . 16
8.6 How Text Guides an Image . . . 16
8.7 Classifier-Free Guidance: The Intuition . . . 17
8.8 Diffusion: Advantages and Limitations . . . 17

9 GANs: Generative Adversarial Networks **17**
9.1 Definition and Intuition . . . 17
9.2 The Original Minimax Objective . . . 17
9.3 How GAN Training Proceeds . . . 18
9.4 Mode Collapse and Training Instability . . . 18
9.5 GANs Compared with Diffusion Models . . . 18

10 Comparing GPT, DALL-E, Codex, and Stable Diffusion **19**

11 Important Related Topic: Retrieval-Augmented Generation (RAG) **19**
11.1 What Is RAG? . . . 19
11.2 A Typical RAG Pipeline . . . 20

11.3 Why RAG Helps, and Why It Is Not a Complete Solution 20

12 Fine-Tuning, Prompting, and RAG: What Is the Difference? 20

13 Enterprise AI: Use Cases Across Industries 21

13.1 Banking and Financial Services 21
13.2 Insurance 21
13.3 Healthcare and Life Sciences 21
13.4 Retail and E-Commerce 22
13.5 Manufacturing and Industrial Operations 22
13.6 IT Services and Software Engineering 22
13.7 Telecommunications 23
13.8 Education and Research 23
13.9 Human Resources and Corporate Functions 23
13.10 Logistics and Supply Chain 23
13.11 Energy, Utilities, and Sustainability 24
13.12 Media, Marketing, and Design 24

14 From a Demo to a Reliable Enterprise System 24

14.1 A Practical Architecture 24
14.2 Build Versus Buy 25
14.3 Evaluation: How Do We Know It Works? 25
14.4 A Simple Business Case 25

15 Risks, Responsible Use, and Governance 26

15.1 Hallucination and Factual Errors 26
15.2 Privacy and Confidentiality 26
15.3 Security Risks 26
15.4 Bias, Fairness, and Accessibility 26
15.5 Copyright, Provenance, and Disclosure 27
15.6 Human Oversight 27

16 A Worked Example: Building an Internal Policy Assistant 27

16.1 Requirements 27
16.2 Design 27
16.3 Example Interaction 27
16.4 How to Evaluate the Pilot 28

17 A Practical Study Plan 28

18 Glossary of Important Terms **28**

19 Self-Assessment Questions **30**

20 Conclusion **30**

1 How to Use This Document

This document is written from the point of view of a learner who is building an understanding of Generative AI step by step. The aim is not merely to memorise definitions, but to understand what a model receives as input, what happens internally, what the mathematics means, what the model produces, and where it can be useful in a real organisation.

The explanations use plain English and practical examples familiar in an Indian workplace. Mathematical expressions are introduced gradually. You do not need to understand every equation on the first reading; first understand the intuition, then return to the equation and connect each symbol with the explanation.

Learning path:

1. Understand Artificial Intelligence, Machine Learning, Deep Learning, and Generative AI.
2. Learn the basic building blocks: data, parameters, training, loss, probability, and embeddings.
3. Understand the Transformer architecture and how GPT-style models generate text.
4. Learn how image-generation approaches such as GANs and diffusion models work.
5. Understand GPT, DALL-E, Codex, and Stable Diffusion as model families or products.
6. Connect these ideas to enterprise use cases, risks, evaluation, and deployment.

2 The Big Picture: AI, ML, Deep Learning, and Generative AI

2.1 Artificial Intelligence

Artificial Intelligence (AI) is the broad field of building computer systems that perform tasks associated with capabilities such as understanding language, recognising patterns, planning, predicting outcomes, or generating content.

For example, a system that detects fraudulent card transactions, a chatbot that answers employee questions, and a program that recognises defects in a factory image are all AI applications. They do not necessarily use the same kind of model.

2.2 Machine Learning

Machine Learning (ML) is a part of AI in which a system learns patterns from data instead of relying only on manually written rules.

Suppose we want to estimate the selling price of a house. We may provide historical examples containing area, location, number of bedrooms, age of the building, and actual selling price. A model learns relationships between the inputs and the target price. It can then estimate the price of a new house.

A simple supervised-learning setup is

$$\hat{y} = f_\theta(x),$$

where:

- x is the input, such as house area and location;
- y is the known answer in the training data;

- $\hat{y}$ is the model's prediction;
- f_θ is the model;
- θ represents its learnable parameters.

Training adjusts θ so that predictions become more useful according to a chosen loss function.

2.3 Deep Learning

Deep Learning is a branch of ML that uses neural networks with multiple layers. These layers learn representations of data. Early layers may identify simple patterns, while later layers can combine them into more complex patterns.

For images, a network may learn edges and textures before learning object parts. For language, a network can learn relationships among words, phrases, and longer contexts.

2.4 Generative AI

Generative AI refers to systems that learn patterns in existing data and use those patterns to create new content, such as text, code, images, audio, video, or structured outputs.

Examples include:

- generating a first draft of a customer email;
- creating an illustration from a text prompt;
- suggesting a Python function from a natural-language description;
- summarising a long policy document;
- generating synthetic examples for testing a software system.

A generative model does not simply retrieve a memorised answer in every case. It learns a distribution or set of patterns and produces an output that is plausible under what it has learned. However, plausible does not always mean correct.

2.5 Discriminative and Generative Models

A discriminative model typically predicts a label or value from an input. For example, it may estimate whether a transaction is fraudulent:

$$P(y \mid x).$$

A generative model learns a way to model the data itself, such as $P(x)$, or the output conditioned on a prompt, such as $P(y \mid x)$. The distinction is useful but not absolute: conditional generative models also use $P(y \mid x)$, where y may be a sequence of words or an image.

Aspect	Discriminative example	Generative example
Input	Transaction details	Prompt: "Explain this transaction pattern"
Output	Fraud probability or label	A written explanation or synthetic example
Main objective	Predict a target accurately	Model patterns and generate content
Common evaluation	Accuracy, precision, recall, RMSE	Factuality, relevance, quality, safety, task success

3 Core Ideas Needed Before Studying Large Models

3.1 Data, Parameters, and Training

A model is a mathematical function with adjustable parameters. During training, the model sees examples and updates those parameters to reduce a loss. Parameters are not the same as hyperparameters: parameters are learned, while hyperparameters (such as learning rate, batch size, and model depth) are selected by the training team.

A simplified training loop is:

1. Feed a batch of examples into the model.
2. Compute the model's predictions.
3. Compare predictions with the training targets using a loss function.
4. Calculate gradients using backpropagation.
5. Update parameters using an optimiser such as stochastic gradient descent or Adam.
6. Repeat over many batches.

3.2 Loss and Gradient Descent

A loss function measures how far a model's prediction is from the target according to a chosen objective. For regression, mean squared error is common:

$$\mathcal{L}_{\text{MSE}} = \frac{1}{N}\sum_{i=1}^{N}(y_i - \hat{y}_i)^2.$$

Here, N is the number of examples, y_i is the true value, and $\hat{y}_i$ is the prediction. Squaring makes large errors count more heavily.

Gradient descent updates parameters in the direction that reduces the loss:

$$\theta_{t+1} = \theta_t - \eta \nabla_\theta \mathcal{L}(\theta_t),$$

where η is the learning rate and $\nabla_\theta \mathcal{L}$ is the gradient of the loss with respect to the parameters. In plain terms, the gradient indicates how the loss changes when parameters change slightly.

3.3 Probability and the Softmax Function

Language models often assign a score, called a logit, to each possible next token. Softmax converts logits into probabilities:

$$P_i = \frac{e^{z_i}}{\sum_{j=1}^{K} e^{z_j}},$$

where z_i is the logit for candidate i, and K is the number of candidates. The probabilities are non-negative and sum to one.

A model may assign a higher probability to "Delhi" than to "Mumbai" after a particular prompt, but the probability depends on the context and the model. It is not a guarantee that the selected continuation is factually correct.

3.4 Tokens and Tokenisation

A token is a unit processed by a language model. A token may be a whole word, part of a word, punctuation, or another text fragment. Tokenisation converts text into token IDs that the model can process.

For example, a long technical word may be split into multiple tokens. Therefore, one token is not always equal to one word. This matters because context windows, inference cost, and API limits are commonly measured in tokens.

3.5 Embeddings

An embedding represents an item as a vector of numbers. Words, text passages, images, or other objects can be mapped into vector spaces. Similar items may have vectors that are close under a similarity measure, although the exact behaviour depends on how the embedding model was trained.

The dot product of vectors a and b is

$$a \cdot b = \sum_{i=1}^{d} a_i b_i.$$

Cosine similarity is

$$\cos(a, b) = \frac{a \cdot b}{\|a\|_2 \|b\|_2}.$$

It measures the angle-based similarity of two non-zero vectors. Embeddings are important in semantic search, document retrieval, recommendation systems, and Retrieval-Augmented Generation (RAG).

4 Transformers: The Foundation of Modern Language Models

4.1 What Is a Transformer?

A Transformer is a neural-network architecture introduced for sequence modelling. It relies heavily on attention mechanisms, which allow each token to use information from other relevant tokens. Transformers are used in language models, vision models, speech systems, and multimodal systems.

Before Transformers became dominant, recurrent neural networks processed sequences step by step. Transformers allow much of the training computation for a sequence to be parallelised, while attention directly models relationships between positions.

4.2 The Intuition Behind Attention

Consider the sentence: "The engineer placed the sensor beside the turbine because it was overheating." To interpret "it", a model needs to relate that word to other words in the sentence. Attention provides a mechanism for assigning different weights to information from different token positions.

In scaled dot-product attention:

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^\top}{\sqrt{d_k}}\right)V.$$

The terms mean:

- Q (queries): what each token is looking for;
- K (keys): what each token offers for matching;
- V (values): the information that can be passed onward;
- $QK^\top$: similarity scores between queries and keys;
- $\sqrt{d_k}$: a scaling factor that helps keep scores in a numerically useful range;
- softmax: converts scores into attention weights;
- multiplication by V: combines information according to those weights.

A useful mental picture is a question-and-answer desk. Each token asks, "Which other tokens are relevant to me?" The keys help calculate relevance, and the values provide the information that is combined.

4.3 Why Use Multiple Attention Heads?

Multi-head attention runs several attention operations in parallel. Different heads can learn different patterns, such as nearby grammatical relationships, long-distance references, or other statistical dependencies. We should not assume that each head has one clean human-readable purpose, but multiple heads give the model several learned ways to combine information.

A simplified form is

$$\text{MultiHead}(Q, K, V) = \text{Concat}(H_1, \ldots, H_h)W^O,$$

where each H_i is an attention head, h is the number of heads, and W^O is a learned output projection.

4.4 Positional Information

Attention alone does not automatically encode the order of tokens in the way a language model needs. Transformers therefore use positional information, for example positional embeddings or rotary position encodings. This helps the model distinguish sequences such as "the dog chased the cat" and "the cat chased the dog".

4.5 A Simplified Transformer Block

A common Transformer block contains:

1. a self-attention sublayer;
2. a feed-forward neural network applied to each token representation;
3. residual connections that help information and gradients flow;
4. layer normalisation or a related normalisation arrangement.

The exact ordering and implementation differ between model families. A simplified feed-forward layer can be written as

$$\mathrm{FFN}(x) = W_2\,\sigma(W_1 x + b_1) + b_2,$$

where W_1, W_2 are learned matrices, b_1, b_2 are biases, and σ is a nonlinear activation function.

4.6 Encoder, Decoder, and Encoder–Decoder Designs

- **Encoder-only:** builds contextual representations of input text. Such designs are useful for classification, extraction, and representation learning.
- **Decoder-only:** predicts the next token from the preceding context. GPT-style models commonly use this design.
- **Encoder–decoder:** an encoder processes the input and a decoder generates an output conditioned on it. This design has been used for translation, summarisation, and text-to-text tasks.

In decoder-only language models, a causal mask prevents a position from attending to future tokens during next-token training. Otherwise, the model could simply look at the answer it is supposed to predict.

4.7 How a GPT-Style Model Generates Text

A simplified generation cycle is:

1. The prompt is tokenised.
2. Token IDs are mapped to vectors and combined with positional information.
3. Transformer layers process the context.
4. The final representation is converted into logits for the next token.
5. Softmax or another decoding procedure determines the next-token distribution.
6. A decoding strategy selects a token, which is added to the context.
7. The process repeats until a stopping condition is reached.

Autoregressive language modelling factorises the probability of a sequence as

$$P(x_1, x_2, \ldots, x_T) = \prod_{t=1}^{T} P(x_t \mid x_1, \ldots, x_{t-1}).$$

This says that the probability of a whole token sequence can be expressed as a product of next-token conditional probabilities.

Training commonly minimises negative log-likelihood, also called cross-entropy loss for the next-token task:

$$\mathcal{L} = -\sum_{t=1}^{T} \log P_\theta(x_t \mid x_{<t}).$$

The model is penalised when it assigns low probability to the actual next token in the training sequence.

4.8 Decoding: Greedy, Sampling, Temperature, and Top-p

Training and generation are different stages. During generation, a decoding strategy determines how the next token is selected.

- **Greedy decoding:** chooses the highest-probability token at each step. It is simple but may produce repetitive or less varied text.
- **Sampling:** selects a token according to a probability distribution, allowing variation.
- **Temperature:** scales logits before softmax. A lower temperature makes the distribution sharper; a higher temperature generally makes it flatter. It changes randomness, not the underlying knowledge of the model.
- **Top-k:** restricts sampling to the k highest-scoring candidates.
- **Top-p (nucleus sampling):** selects from a smallest set of tokens whose cumulative probability reaches a chosen threshold p.

Temperature scaling can be written as

$$P_i = \frac{\exp(z_i/\tau)}{\sum_j \exp(z_j/\tau)},$$

where $\tau > 0$ is the temperature. These controls can affect output style and diversity, but they do not by themselves ensure factuality or safety.

4.9 What Transformers Do Not Automatically Guarantee

A Transformer can learn impressive patterns, but its architecture does not guarantee truth, reasoning reliability, fairness, privacy, or up-to-date information. A fluent answer can still contain a fabricated source, an incorrect calculation, or an unsupported claim. Reliable applications need evaluation, grounding, access control, and human review appropriate to the risk.

5 GPT: Generative Pre-trained Transformer

5.1 Definition

GPT stands for Generative Pre-trained Transformer. It refers to a family of language models based on Transformer-style autoregressive generation. A GPT-style model is trained to predict tokens from context and can be adapted to follow instructions, answer questions, write, summarise, classify, and assist with many language tasks.

The name can be understood in three parts:

- **Generative:** produces a continuation or output.

- **Pre-trained:** first learns broad patterns from a large training corpus before later adaptation or alignment stages.
- **Transformer:** uses the Transformer architecture or a closely related design.

5.2 How GPT-Style Systems Are Developed

A high-level development pipeline may include:

1. **Pre-training:** learn language patterns by predicting tokens over a large and diverse dataset.
2. **Instruction tuning:** train on examples of instructions and desirable responses so the model is more useful for user requests.
3. **Preference or alignment training:** use human feedback, preference data, or other objectives to encourage helpful and safe behaviour. The precise method varies by system.
4. **Evaluation and safety testing:** test performance, failure modes, robustness, and policy compliance.
5. **Deployment:** serve the model through an application or API, with monitoring and operational controls.

Not every model uses exactly this sequence, and the details of commercial models may not be public.

5.3 What Happens When We Ask GPT a Question?

Suppose an employee asks, "Summarise the travel reimbursement policy and list the documents required." The system tokenises the request, processes it through the model, and generates a response token by token. If the system is connected to an approved policy repository, it may first retrieve relevant policy passages and provide them as context. That retrieval step is an application design choice; it is not automatically present in every language model.

5.4 Strengths and Limitations

Strengths

- Handles many language tasks through a common interface.
- Can summarise, draft, translate, explain, and transform text.
- Can help with code, structured extraction, and analysis when properly checked.
- Can be combined with search, databases, APIs, and workflow tools.

Limitations

- May generate confident but incorrect statements (often called hallucinations).
- May reflect biases or gaps present in training and feedback data.
- May struggle with exact arithmetic, long multi-step tasks, or ambiguous instructions.
- May not know recent events unless connected to an appropriate current source.
- Can expose sensitive information if an application is designed or configured poorly.

5.5 Prompting: Giving Clear Instructions

A prompt is the input that guides the model. A useful prompt usually states the task, context, constraints, desired format, and criteria for a good answer.

For example, instead of writing "Explain sales", an employee might write:

> You are assisting an Indian retail operations team. Summarise the attached monthly sales report for a regional manager. Highlight the three largest changes, distinguish facts from possible explanations, use INR, and end with three recommended follow-up checks. Do not invent missing figures.

Prompting can improve consistency, but it cannot replace access to reliable source data or testing.

6 DALL-E: Generating Images from Text

6.1 Definition

DALL-E is a family of text-to-image generative systems associated with OpenAI. A user provides a text description and the system generates an image intended to match the prompt. Capabilities and underlying architectures can differ across generations, so it is better not to assume that every DALL-E version uses the same internal design.

Example prompt: "Create a clean isometric illustration of a solar-powered water purification unit in a rural Rajasthan village, with labelled components and a warm daylight setting." The model attempts to translate the words into visual content.

6.2 How Text-to-Image Generation Works at a High Level

A typical text-to-image system has two broad jobs:

1. **Understand the prompt:** convert the text into a representation that captures concepts, attributes, and relationships.
2. **Generate an image:** use a learned generative process to produce pixels or an image representation consistent with the prompt.

Historically, text-to-image systems have used different approaches, including autoregressive token generation, diffusion-based generation, and other learned image-generation pipelines. The exact implementation is model-specific.

6.3 What Makes Text-to-Image Difficult?

The prompt may contain many simultaneous requirements: objects, counts, colours, relative positions, style, lighting, text labels, and physical relationships. The system must produce a coherent image that satisfies these constraints. It may get the overall concept right but misplace an object, render text incorrectly, or omit a detail.

For enterprise use, the generated image should be reviewed for factual correctness, brand compliance, rights, accessibility, and suitability for the intended audience. Images of technical equipment, medical procedures, safety instructions, or product components need special care because visual plausibility is not proof of engineering accuracy.

7 Codex: AI Assistance for Software Development

7.1 Definition

Codex has referred to OpenAI systems designed for programming-related tasks. Such systems can interpret natural-language instructions and work with code: explaining a function, generating a code draft, suggesting tests, translating between languages, or helping debug an error. The name has been used across different generations and products, so capabilities and implementation should be checked for the particular Codex offering being discussed.

7.2 How a Code Model Works

A code model learns patterns from programming-language text and related context. Given a prompt and existing code, it predicts a continuation or produces a response in code or natural language. Since code has syntax, APIs, types, dependencies, and execution behaviour, a plausible-looking output can still fail to compile or behave incorrectly.

A useful development workflow is:

1. Describe the intended behaviour and constraints.
2. Provide relevant code, error messages, and environment details.
3. Ask for a minimal, explainable change rather than an unnecessary rewrite.
4. Review the proposed diff.
5. Run unit tests, integration tests, linters, and security checks.
6. Test edge cases and inspect any dependency or permission changes.

7.3 Example

Suppose a developer has a FastAPI endpoint that reads book records from PostgreSQL but returns an internal server error. A coding assistant may help inspect connection handling, SQL queries, exception handling, response schemas, and tests. The developer must still run the application and tests, confirm database permissions, and verify that no credentials or private records are exposed.

7.4 Risks of AI-Generated Code

- Incorrect assumptions about libraries, versions, or APIs.
- SQL injection, insecure authentication, or accidental secret exposure.
- Missing error handling and edge cases.
- Licence or provenance concerns in some development settings.
- Tests that pass for a narrow example but fail in production.

AI coding tools should be treated as capable assistants, not as substitutes for code review, secure development practices, or accountability.

8 Stable Diffusion and Latent Diffusion Models

8.1 Definition

Stable Diffusion is a family of image-generation models based on latent diffusion techniques. A prompt guides the generation of an image, while much of the iterative generation is performed in a compressed latent representation rather than directly over every pixel. This can reduce computational cost compared with performing the full process in pixel space.

8.2 What Is a Diffusion Model?

A diffusion model learns to generate data by reversing a gradual noising process. During training, noise is added to a real example over multiple steps. A neural network learns to estimate the noise or otherwise predict a quantity that allows the example to be denoised. During generation, the model starts from random noise and repeatedly refines it toward a sample that resembles the training distribution.

The intuition is similar to starting with television static and repeatedly cleaning it up, except the denoising steps are learned from data and, in text-conditioned systems, guided by the prompt.

8.3 Forward Diffusion: Adding Noise

Let x_0 be a clean training image. A simplified forward process gradually adds Gaussian noise:

$$q(x_t \mid x_{t-1}) = \mathcal{N}\left(x_t; \sqrt{1-\beta_t}\, x_{t-1}, \beta_t I\right),$$

where:

- x_t is the noisy image at step t;
- β_t controls the amount of noise added at that step;
- I is the identity covariance matrix;
- $\mathcal{N}$ denotes a Gaussian distribution.

As t increases, the image becomes increasingly noisy. The schedule of β_t values determines how noise is added over the steps.

A useful closed-form expression is

$$x_t = \sqrt{\bar{\alpha}_t}\, x_0 + \sqrt{1-\bar{\alpha}_t}\, \epsilon, \qquad \epsilon \sim \mathcal{N}(0, I),$$

where $\alpha_t = 1 - \beta_t$ and

$$\bar{\alpha}_t = \prod_{s=1}^{t} \alpha_s.$$

This equation says that a noisy training sample can be formed directly from the clean image and a noise sample, using coefficients determined by the noise schedule.

8.4 Reverse Diffusion: Learning to Denoise

The model learns to estimate the noise ϵ present in a noisy example. A common simplified training objective is

$$\mathcal{L}_{\text{diff}} = \mathbb{E}_{x_0,t,\epsilon}\left[\|\epsilon - \epsilon_\theta(x_t, t, c)\|_2^2\right],$$

where:

- x_0 is a real training image;
- t is a randomly selected diffusion step;
- ϵ is the noise actually added;
- ϵ_θ is the neural network's noise prediction;
- c is optional conditioning information, such as a text representation.

The objective penalises the difference between the actual noise and the predicted noise. At generation time, the model starts with noise and applies a sequence of reverse updates. Each update uses the model's prediction to move toward a less noisy sample. The precise sampler and prediction target can vary across implementations.

8.5 Why Work in Latent Space?

A high-resolution image may contain millions of pixel values. A learned encoder can compress the image into a smaller latent tensor, and a decoder can reconstruct an image from that latent representation.

A simplified latent diffusion pipeline is:

1. Encode a training image into a latent representation z_0.
2. Add noise to z_0 during training.
3. Train a denoising network to predict noise in the latent space, optionally conditioned on text.
4. At generation time, start from a random latent.
5. Iteratively denoise the latent while using the prompt as conditioning.
6. Decode the final latent into an image.

The key benefit is computational efficiency: the denoising network works on a compressed representation. Compression can also discard some fine details, and the quality depends on the encoder, decoder, denoiser, training data, and sampling method.

8.6 How Text Guides an Image

A text encoder converts the prompt into numerical representations. The denoising network uses these representations to guide image formation. Cross-attention is one mechanism that allows spatial or latent features to attend to text features.

For example, for the prompt "a red bicycle beside a blue wall", the model needs to represent the bicycle, its colour, the wall, and the spatial relation "beside". It may produce an approximate interpretation, but detailed counting, exact text, and complicated spatial relations can remain difficult.

8.7 Classifier-Free Guidance: The Intuition

Some diffusion systems combine a conditional prediction with an unconditional prediction. A common form is

$$\hat{\epsilon} = \epsilon_{\text{uncond}} + s\left(\epsilon_{\text{cond}} - \epsilon_{\text{uncond}}\right),$$

where s is the guidance scale. Increasing s often makes the output follow the prompt more strongly, but excessive guidance can reduce diversity or create artefacts. The practical behaviour depends on the model and sampler.

8.8 Diffusion: Advantages and Limitations

Advantages

- Can produce high-quality, diverse images.
- Supports conditioning on text and, in some systems, images or other inputs.
- Iterative sampling provides a controllable generation process.
- Latent diffusion can reduce computation compared with pixel-space diffusion.

Limitations

- Generation can require multiple denoising steps and significant compute.
- Fine details, exact counts, and written text may be wrong.
- Training data can introduce bias or raise copyright and consent concerns.
- Prompt adherence is not a guarantee of factual or technical correctness.

9 GANs: Generative Adversarial Networks

9.1 Definition and Intuition

A Generative Adversarial Network (GAN) contains two neural networks trained in competition:

- **Generator G:** converts a random latent vector z into a synthetic sample $G(z)$.
- **Discriminator D:** estimates whether a sample comes from real training data or from the generator.

Imagine a counterfeiter and a detective. The generator tries to create convincing samples, while the discriminator tries to distinguish generated samples from real ones. As each improves, the other receives a stronger learning signal. This is an analogy only; the networks optimise mathematical objectives rather than having intentions.

9.2 The Original Minimax Objective

A standard GAN objective is

$$\min_G \max_D V(D,G) = \mathbb{E}_{x\sim p_{\text{data}}(x)}[\log D(x)] + \mathbb{E}_{z\sim p_z(z)}[\log(1 - D(G(z)))].$$

Interpretation:

- x is a real sample drawn from the training distribution p_{data}.
- z is a random latent vector drawn from a simple distribution p_z.
- $D(x)$ is the discriminator's estimate that a sample is real.
- $G(z)$ is a generated sample.
- The discriminator tries to classify real and generated samples correctly.
- The generator tries to make generated samples difficult to distinguish from real ones.

This is a minimax game because the networks have competing objectives. In practice, training often uses modified losses and optimisation techniques.

9.3 How GAN Training Proceeds

1. Draw a batch of real examples.
2. Draw random latent vectors and use the generator to create synthetic examples.
3. Update the discriminator to distinguish real examples from generated examples.
4. Update the generator so its outputs receive a more realistic score from the discriminator.
5. Repeat while monitoring quality, diversity, and training stability.

9.4 Mode Collapse and Training Instability

A well-known GAN problem is **mode collapse**: the generator may produce a limited variety of outputs instead of representing the diversity of the data. Training can also be unstable because the two networks must improve in balance. If the discriminator becomes too strong or the learning dynamics become poorly balanced, the generator may receive an unhelpful learning signal.

Techniques such as Wasserstein GAN objectives, gradient penalties, spectral normalisation, careful architecture choices, and balanced update schedules can improve stability, although no method removes every difficulty.

9.5 GANs Compared with Diffusion Models

Aspect	**GAN**	**Diffusion model**
Main idea	Generator competes with discriminator	Learn to reverse a noising process
Generation	Often one generator pass after latent sampling	Usually multiple denoising steps
Training challenge	Adversarial instability and mode collapse	Training and sampling compute; sampler choices
Diversity	Can be limited by mode collapse	Often strong diversity, depending on setup
Historical use	Image synthesis, translation, super-resolution	Text-to-image, image editing, audio and other generation

Neither family is universally best. The appropriate choice depends on the task, quality requirements, latency, compute budget, data, and deployment constraints.

10 Comparing GPT, DALL-E, Codex, and Stable Diffusion

The names are not four equivalent architecture types. They refer to model families or systems aimed at different tasks.

Name	Main purpose	Input and output	Important point
GPT	General language generation and understanding	Text or other supported inputs; generates text or structured responses	GPT-style models commonly use autoregressive Transformer designs.
DALL-E	Text-guided image generation	Text prompt; generates an image	A product/model family, not one universal architecture across all versions.
Codex	Programming assistance	Natural-language instructions and code context; generates or analyses code	Generated code must be reviewed, tested, and security-checked.
Stable Diffusion	Image generation and editing	Text and, in some workflows, image conditioning; generates or modifies images	Known for latent diffusion approaches; implementations and licences vary.

A helpful distinction is:

- **Architecture** describes the design pattern, such as a Transformer, GAN, or diffusion process.
- **Model** is a trained set of parameters using an architecture and training procedure.
- **Product or model family** is a named offering that may include multiple model versions and tools.
- **Application** is the system that people actually use, including the model, interface, data sources, permissions, and workflow logic.

For instance, an enterprise application might use a GPT-style model to understand a request, an embedding model to retrieve internal policy passages, and a separate image model to generate a visual draft. These are components of one solution, not one single model.

11 Important Related Topic: Retrieval-Augmented Generation (RAG)

11.1 What Is RAG?

Retrieval-Augmented Generation combines a generative model with a retrieval system. Instead of relying only on information encoded during training, the application retrieves relevant material

from an approved knowledge source and gives it to the model as context.

For example, a company may have thousands of pages of HR policies. When an employee asks about parental leave, the application searches the policy collection, retrieves the relevant passages, and asks the language model to answer using those passages.

11.2 A Typical RAG Pipeline

1. **Ingest documents:** collect approved files, web pages, or database records.
2. **Chunk documents:** split long content into manageable sections while preserving useful context.
3. **Create embeddings:** convert chunks into vectors.
4. **Index:** store vectors and metadata in a search index or vector database.
5. **Retrieve:** embed the user's query and retrieve relevant chunks using semantic, keyword, or hybrid search.
6. **Construct context:** provide retrieved text and instructions to the language model.
7. **Generate and cite:** produce an answer grounded in the retrieved sources, ideally with traceable citations.
8. **Evaluate:** measure retrieval relevance, answer correctness, citation quality, and access-control behaviour.

11.3 Why RAG Helps, and Why It Is Not a Complete Solution

RAG can improve freshness and source traceability because the model receives relevant documents at answer time. However, it can still fail if retrieval misses the correct document, documents are outdated, permissions are incorrect, or the model misinterprets the retrieved content. RAG is not the same as retraining or fine-tuning a model.

12 Fine-Tuning, Prompting, and RAG: What Is the Difference?

Approach	What changes?	Good fit
Prompting	Instructions and context supplied at inference time	Quick task adaptation, formatting, and clear task guidance
RAG	External information retrieved and supplied at inference time	Internal policies, product catalogues, technical manuals, and frequently changing knowledge
Fine-tuning	Model parameters are further trained on a task-specific dataset	Consistent task behaviour, specialised formats, or domain patterns when sufficient high-quality examples exist

These approaches can be combined. For example, a fine-tuned model can use RAG to answer from current internal documents. The choice should be based on measured requirements, not on the assumption that a more complex solution is automatically better.

13 Enterprise AI: Use Cases Across Industries

Enterprise AI means applying AI within an organisation's real business processes. The objective is not simply to add a chatbot. A good solution should improve a measurable outcome such as turnaround time, service quality, cost, accuracy, risk detection, or employee productivity.

13.1 Banking and Financial Services

Potential applications include:

- **Customer support:** answer common questions about products, fees, and service processes using approved information.
- **Document processing:** extract fields from forms, statements, and supporting documents for human verification.
- **Fraud and anomaly analysis:** prioritise suspicious patterns for investigation. Traditional ML and graph methods may be more suitable for the detection model, while generative AI can help summarise case evidence.
- **Compliance assistance:** search regulations and internal controls, compare documents, and prepare draft reports with citations.
- **Relationship-manager support:** summarise permitted customer history and draft follow-up communications.

Important controls: customer-data privacy, auditability, explainability where required, human approval for consequential decisions, and strict controls on financial advice or transactions.

13.2 Insurance

- Extract information from claim forms, invoices, and policy documents.
- Summarise claim histories for adjusters.
- Search policy wording and explain relevant clauses.
- Assist with drafting customer communications.
- Detect unusual claim patterns using predictive models, with investigators making the final judgement.

The model should not independently reject a claim merely because a generated explanation sounds convincing. Policy rules, evidence, regulation, and human review must govern high-impact decisions.

13.3 Healthcare and Life Sciences

- Summarise medical literature or clinical documentation for qualified professionals.
- Help researchers search scientific papers and organise evidence.
- Draft administrative notes and patient-friendly explanations for clinician review.
- Support drug discovery research, molecular analysis, and laboratory workflow documentation.

- Assist with appointment scheduling and non-diagnostic administrative questions.

Healthcare use requires particularly strong privacy and safety controls. A general-purpose model is not a substitute for a clinician, and generated medical statements require verification against reliable evidence and relevant professional standards.

13.4 Retail and E-Commerce

- Product-description generation and catalogue enrichment.
- Customer-service assistants that retrieve order and policy information through authorised tools.
- Search that understands intent rather than relying only on exact keywords.
- Demand forecasting, inventory optimisation, and product recommendations using suitable predictive models.
- Drafting marketing variants and adapting content to different customer segments, subject to brand and privacy rules.

A useful distinction is that a forecasting model predicts quantities, while a generative model writes explanations or creates marketing material. A system can combine both.

13.5 Manufacturing and Industrial Operations

- Generate first drafts of maintenance instructions from approved manuals.
- Search equipment logs and summarise recurring faults.
- Use computer vision to identify visible defects on production lines.
- Use sensor-based ML to predict possible equipment failure.
- Help engineers compare design options or draft simulation documentation.

For safety-critical machinery, generated instructions must be checked against validated procedures. An AI system should not be given unrestricted control over industrial equipment simply because it can produce a plausible action plan.

13.6 IT Services and Software Engineering

- Explain unfamiliar code and legacy systems.
- Generate test cases, boilerplate code, and documentation drafts.
- Summarise incidents and suggest troubleshooting steps from approved runbooks.
- Assist with log analysis and ticket classification.
- Support migration planning and code translation.

Useful metrics include defect rate, escaped bugs, review time, test coverage, change-failure rate, and developer satisfaction—not just the number of lines of code generated.

13.7 Telecommunications

- Summarise customer complaints and suggest next steps.
- Assist support teams with network troubleshooting documentation.
- Forecast traffic and detect anomalies using time-series models.
- Generate plain-language explanations of service plans.
- Help field teams retrieve approved installation and maintenance procedures.

Customer records and network details must be protected, and generated troubleshooting steps should be checked before being applied to live infrastructure.

13.8 Education and Research

- Create practice questions and explanations at different difficulty levels.
- Help search research literature and organise references.
- Provide feedback on writing structure, while preserving academic integrity.
- Assist researchers with code drafts, data-cleaning scripts, and experiment documentation.
- Create accessible summaries of complex material.

Generated references, quotations, calculations, and scientific claims must be verified. A citation-looking string is not proof that a paper exists or supports the claim.

13.9 Human Resources and Corporate Functions

- Answer employee questions using approved HR policies.
- Summarise internal procedures and draft routine communications.
- Extract fields from onboarding forms with employee confirmation.
- Help employees find training material and explain internal processes.
- Assist with procurement, legal-document review, and finance reporting drafts.

Recruitment, promotion, compensation, and performance decisions can significantly affect people. AI use in these areas requires careful fairness testing, transparency, lawful processing, and appropriate human decision-making.

13.10 Logistics and Supply Chain

- Predict demand and estimate delivery delays using historical data.
- Summarise shipment exceptions and suggest investigation steps.
- Extract information from bills, purchase orders, and delivery documents.
- Support warehouse staff with natural-language search over procedures.
- Generate scenario summaries for planners comparing supply-chain options.

Optimisation and forecasting models are often central to logistics decisions. Generative AI can make their outputs easier to query and explain, but it should not replace validated optimisation logic without evidence.

13.11 Energy, Utilities, and Sustainability

- Forecast demand or renewable-energy output using time-series models.
- Summarise equipment inspection records and maintenance logs.
- Search environmental reports and regulatory requirements.
- Assist with drafting sustainability disclosures from verified source data.
- Support engineering analysis by organising simulation outputs and documentation.

Environmental reporting needs traceable calculations and source data. A generated narrative must not substitute for measured emissions, validated assumptions, or required assurance.

13.12 Media, Marketing, and Design

- Draft campaign concepts, headlines, and audience-specific variants.
- Generate storyboards, illustrations, and early design explorations.
- Summarise campaign feedback and categorise comments.
- Produce first drafts of video scripts, captions, and internal briefs.

Brand consistency, copyright, consent, disclosure requirements, and human creative direction remain important. Generated images should not be treated as accurate evidence of a real event.

14 From a Demo to a Reliable Enterprise System

14.1 A Practical Architecture

A production application may contain several components:

1. **User interface:** web application, mobile application, or internal chat interface.
2. **Authentication and authorisation:** verifies who the user is and what they may access.
3. **Application backend:** validates requests and manages business logic.
4. **Model endpoint:** calls the selected model through a controlled interface.
5. **Retrieval or tools:** searches documents or invokes approved APIs where needed.
6. **Guardrails:** enforce input checks, output validation, data policies, and tool permissions.
7. **Logging and monitoring:** capture appropriate operational information without unnecessarily retaining sensitive content.
8. **Evaluation and human review:** measure quality and route risky or uncertain cases for review.

The model should generally not be given direct, unrestricted access to every company database or system. A backend should expose narrowly scoped operations, enforce permissions, validate arguments, and record consequential actions.

14.2 Build Versus Buy

An organisation can use a commercial model API, deploy an open-weight model, or combine several options. The decision should consider:

- quality on the organisation's actual tasks;
- latency, throughput, and availability;
- inference and infrastructure cost;
- data residency, privacy, and contractual obligations;
- licence terms and deployment rights;
- ability to customise, evaluate, and monitor;
- integration effort and vendor dependency.

Open weights do not automatically mean that all training data is public, that every use is permitted, or that the model is free to operate. The applicable licence and deployment terms must be checked.

14.3 Evaluation: How Do We Know It Works?

Evaluation should be designed around the task. Possible measures include:

- **Correctness:** is the answer supported by the evidence?
- **Groundedness:** does the answer stay within the retrieved or supplied information?
- **Retrieval quality:** did the system find the right documents?
- **Task completion:** did the user successfully finish the intended workflow?
- **Safety:** does the system resist inappropriate requests and unsafe actions?
- **Latency and cost:** how long does it take, and what is the cost per successful task?
- **Robustness:** does it work across different wording, edge cases, and realistic inputs?
- **Fairness and accessibility:** are performance gaps or barriers present for relevant user groups?

For a customer-support assistant, for example, a high answer-similarity score is not enough. We should also test whether the answer reflects the current policy, whether citations support it, whether the system respects the customer's permissions, and whether escalation occurs when necessary.

14.4 A Simple Business Case

A pilot should have a baseline and a target. Suppose employees currently spend an average of 12 minutes finding an answer in internal documents. A proposed assistant may reduce the average

time to 7 minutes. The gross time saving is 5 minutes per successful query, but the business case should also account for review time, failed answers, maintenance, inference cost, and the value of employees' time.

A simplified annual net-benefit model is

$$\text{Net Benefit} = \text{Value of Time Saved} + \text{Other Measured Benefits} - \text{Total Operating Cost}.$$

Return on investment can be estimated as

$$\text{ROI} = \frac{\text{Benefits} - \text{Costs}}{\text{Costs}} \times 100\%.$$

The numbers should come from a measured pilot, not from optimistic assumptions. Benefits that are difficult to quantify should be stated separately rather than presented as guaranteed savings.

15 Risks, Responsible Use, and Governance

15.1 Hallucination and Factual Errors

A model may produce a fluent answer that is unsupported or wrong. Common mitigations include retrieval from approved sources, citations, constrained output schemas, calculations performed by deterministic code, confidence-aware escalation, and human review for consequential tasks. None is a complete guarantee.

15.2 Privacy and Confidentiality

Prompts may contain personal data, customer records, source code, commercial information, or credentials. Organisations should define what information may be sent to each model, review provider terms and retention settings, minimise data collection, redact secrets where appropriate, and enforce role-based access controls.

15.3 Security Risks

Prompt injection occurs when untrusted content attempts to influence a model or tool-using application to ignore intended instructions. Retrieved documents, emails, web pages, and user-provided files must be treated as data rather than trusted instructions. Tool calls should be authorised by application code, not merely by the model's statement that an action is safe.

Other risks include insecure output handling, excessive permissions, data leakage, malicious code generation, and abuse of automated workflows. Security testing should include realistic adversarial cases.

15.4 Bias, Fairness, and Accessibility

Training data and system design can produce uneven performance across languages, dialects, demographic groups, or contexts. Evaluation should use relevant and lawful test cases. A system that works well in standard written English may perform differently with Indian English, Hinglish, regional-language text, or domain-specific terminology.

15.5 Copyright, Provenance, and Disclosure

Organisations should review applicable law, licences, contractual terms, and internal policies before using generated content commercially. They should consider whether source materials, generated assets, or customer-facing outputs need attribution or disclosure. Legal requirements differ by jurisdiction and use case.

15.6 Human Oversight

Human oversight should be proportional to risk. A draft of an internal meeting agenda may need only light review. A medical recommendation, credit decision, legal filing, or industrial control instruction requires stronger validation and accountable human decision-making.

16 A Worked Example: Building an Internal Policy Assistant

Suppose a company wants employees to ask questions about travel, leave, and reimbursement policies.

16.1 Requirements

- Answer only from approved and current policy sources.
- Provide citations or links to relevant policy passages.
- Respect access permissions.
- Say when the source does not contain the answer.
- Escalate ambiguous or high-impact cases to HR.

16.2 Design

1. Store approved policy documents in a controlled repository.
2. Extract text and split it into sections with document title, version, and effective date metadata.
3. Create embeddings and a searchable index.
4. At query time, authenticate the employee and retrieve only documents they are allowed to see.
5. Supply the question and retrieved passages to a language model with instructions to answer from the evidence.
6. Return the answer with source links and a clear statement when evidence is insufficient.
7. Log operational metrics and evaluate a representative set of questions.

16.3 Example Interaction

Employee: “Can I claim a hotel expense above the standard limit when travelling for a client project?”

Assistant behaviour: It should retrieve the current travel policy and any approved exception procedure. If the policy states that a manager's prior approval is required, the answer should explain that requirement and cite the relevant section. If the available documents do not specify the applicable limit or exception, the assistant should not invent a number; it should direct the employee to the policy owner or HR.

16.4 How to Evaluate the Pilot

Create a test set covering straightforward questions, exceptions, outdated documents, conflicting policies, questions with no answer, and attempts to access restricted material. Ask qualified reviewers to grade correctness, source support, clarity, and escalation behaviour. Compare performance with the current process and measure time saved, user satisfaction, and operational cost.

17 A Practical Study Plan

A learner can study the material in the following order. The times are illustrative; adjust them according to your background.

Stage	Focus	Practice task
1	Probability, vectors, matrices, derivatives, gradient descent	Calculate a softmax distribution and one small gradient-descent update.
2	Neural networks and training	Train a small classifier and inspect its loss curve.
3	Tokenisation, embeddings, attention, Transformers	Draw a Transformer block and explain the role of Q, K, and V.
4	GPT-style generation	Compare greedy decoding with sampling on the same prompt.
5	GANs and diffusion	Explain the generator/discriminator game and the forward/reverse diffusion process.
6	Text-to-image systems	Test how changes in prompt detail affect the generated result.
7	RAG and enterprise architecture	Build a small question-answering system over a few approved documents.
8	Evaluation and governance	Create a test set, record failures, and define when human review is needed.

18 Glossary of Important Terms

AI
The broad field of building systems that perform tasks associated with intelligent behaviour.

Machine Learning
Methods that learn patterns from data.

Deep Learning
ML using neural networks with multiple layers.

Generative model
A model that learns patterns in data and can generate new samples or outputs.

Parameter
A learned numerical value inside a model.

Loss function
A numerical objective used to measure training error.

Gradient
The direction and rate of change of a function with respect to its variables.

Token
A unit of text or other data processed by a model.

Embedding
A numerical vector representation of an item.

Attention
A mechanism for weighting information from different positions or inputs.

Transformer
A neural-network architecture built around attention and feed-forward layers.

Autoregressive
Generating a sequence one element at a time, conditioned on earlier elements.

GAN
A generator and discriminator trained with competing objectives.

Diffusion
A generative approach that learns to reverse a gradual noising process.

Latent space
A learned, often compressed representation of data.

Fine-tuning
Further training a pre-trained model on a more specific dataset or objective.

RAG
Retrieval-Augmented Generation; retrieving external material to ground a model's answer.

Inference
Using a trained model to produce an output.

Hallucination
A generated claim or detail that is unsupported or incorrect.

Guardrail
A control designed to restrict unsafe, invalid, or unauthorised system behaviour.

19 Self-Assessment Questions

Use these questions to check whether the concepts are understood rather than merely memorised.

1. What is the difference between AI, ML, deep learning, and Generative AI?
2. Why is next-token prediction useful for training a language model?
3. In the attention equation, what roles do queries, keys, and values play?
4. Why does a decoder-only language model use a causal mask?
5. What is the difference between a model's training objective and its decoding strategy?
6. How does a diffusion model learn to generate an image from random noise?
7. Why can latent diffusion be more computationally efficient than pixel-space diffusion?
8. What are the roles of the generator and discriminator in a GAN?
9. Explain mode collapse in your own words.
10. How do GPT, DALL-E, Codex, and Stable Diffusion differ in their intended tasks?
11. When might RAG be more suitable than fine-tuning?
12. Why should AI-generated code be tested even when it looks correct?
13. Name three useful enterprise AI use cases and one risk for each.
14. What should a company measure in an AI pilot before scaling it?
15. Why is human oversight especially important in high-impact applications?

20 Conclusion

Generative AI is not one single technique. GPT-style language models, text-to-image systems, code assistants, GANs, and diffusion models solve related but different problems. Transformers are a major architecture for modelling sequences and context; diffusion models generate samples by learning to reverse noise; GANs use a competitive training setup between a generator and discriminator.

For enterprise use, model capability is only one part of the solution. Data quality, retrieval, permissions, evaluation, security, user experience, operating cost, and human accountability determine whether a promising demonstration becomes a dependable business tool. The best starting point is a clearly defined problem, a small measurable pilot, and a careful review of both benefits and failure cases.

References

[1] A. Vaswani et al., "Attention Is All You Need," *Advances in Neural Information Processing Systems*, 2017. https://arxiv.org/abs/1706.03762

[2] A. Radford et al., "Improving Language Understanding by Generative Pre-Training," OpenAI, 2018. https://cdn.openai.com/research-covers/language-unsupervised/language_understanding_paper.pdf

[3] I. Goodfellow et al., "Generative Adversarial Nets," *Advances in Neural Information Processing Systems*, 2014. https://arxiv.org/abs/1406.2661

[4] J. Ho, A. Jain, and P. Abbeel, "Denoising Diffusion Probabilistic Models," 2020. https://arxiv.org/abs/2006.11239

[5] R. Rombach et al., "High-Resolution Image Synthesis with Latent Diffusion Models," 2022. https://arxiv.org/abs/2112.10752

[6] P. Lewis et al., "Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks," 2020. https://arxiv.org/abs/2005.11401

[7] L. Ouyang et al., "Training Language Models to Follow Instructions with Human Feedback," 2022. https://arxiv.org/abs/2203.02155

[8] OpenAI, "DALL-E: Creating Images from Text." https://openai.com/research/dall-e/

[9] Stability AI, "Stable Diffusion." https://stability.ai/

[10] National Institute of Standards and Technology, "Artificial Intelligence Risk Management Framework (AI RMF 1.0)," 2023. https://www.nist.gov/itl/ai-risk-management-framework